

Project Kick-Off: Temperature Swing Integration

Project Context

The Temperature Cabinet Control project has successfully achieved its original objective of allowing the temperature cabinet setpoint to be controlled programmatically from the DLS/CODESYS system.

As an additional achievement, remote cabinet start/stop control has also been successfully demonstrated while retaining the existing physical operator controls. Well done on getting this stage working.

The next stage is to use this functionality within the existing R&D DLS architecture to create a new automated Temperature Swing program.

Project Objective

Create a new CODESYS program called Temperature Swing which allows a technician to configure and start a controlled temperature swing from the HTML HMI.

One execution of the program should perform:

- current temperature → requested temperature setpoint.

Once the technician presses Start, the DLS should manage the temperature cabinet and, where requested, the test pressure until the selected temperature measurement reaches the requested setpoint and satisfies the API 6A temperature stabilisation requirement.

When to Work on This Project

This project should be undertaken alongside your current project, particularly when you cannot access the temperature cabinets because they are being used for R&D or other testing.

Live testing takes priority. Lack of cabinet access should not automatically stop progress: CODESYS work, HTML HMI development, investigation, documentation and offline testing should continue where possible.

Do not interrupt or interfere with existing testing to progress this project.

Development Environment

The following are the agreed and required starting points for this project.

CODESYS Project

Project: V:\Mechatronics\Projects\2025-26\DLS Temperature Cabinet Control\Temperature Swing Integration\CODESYS Project\Omkar_Temperature_Swing_Integration.project

This is the CODESYS development project for the Temperature Swing integration. Do not create a replacement CODESYS project.

GitHub

Repository: tlealean/RnD

Branch: Omkar_Temperature_Swing_Integration

- all GitHub development for this project must take place on this branch;
- do not create a separate repository for this work;

- do not work directly on main;
- do not merge this branch into main.

Any eventual integration into the main R&D codebase will be reviewed separately.

Specification

API 6A copy: V:\Mechatronics\Projects\2025-26\DLS Temperature Cabinet Control\Temperature Swing Integration\Specification\API 6A.pdf

Review the relevant requirements before designing the temperature-control logic, particularly:

- F.1.9 – Temperature Testing;
- F.1.10 – Hold Periods / Temperature Stabilisation;
- F.1.11 – Pressure and Temperature Cycles.

For this program, the important temperature requirements include:

- the requested temperature extreme must be reached or passed in the correct direction;
- temperature is considered stabilised when its rate of change is less than 0.5 °C/min;
- the temperature should not exceed the required extreme by more than 11 °C.

There is no requirement in this task to create a specific heating or cooling ramp rate. Do not invent one.

General Design Principle

The existing DLS system already performs many similar operations. Before writing new functionality, investigate what already exists and reuse it wherever practical.

Examples include:

- program selection and state-machine structure;
- Start Dialog behaviour and delayed start;
- CSV/data recording;
- pressure application and pressure monitoring;
- solenoid control;
- program prompts;
- stabilisation calculations;
- channel selection;
- Pressure Display status indication.

The aim is for Temperature Swing to behave like an existing DLS program rather than feel like a separate application added onto it. Consistency is important for technicians operating the rig.

Temperature Swing Start Dialog

Create the required Start Dialog configuration for the new Temperature Swing program within the HTML HMI. The layout and behaviour should follow the existing Start Dialog conventions.

Retain the relevant existing fields, including:

- Test Pressure;
- Test Section Number;
- Test Name;
- Delayed Start;
- Readings per Second;
- Cycles – fixed to 1.

Temperature Setpoint

Add a Temperature Setpoint field in °C. It must support positive temperatures, negative temperatures and decimal values where supported by the cabinet interface.

Do not introduce one universal hard-coded cabinet temperature range at this stage because different cabinets have different capabilities.

Temperature Monitoring Channel

Provide a dropdown using behaviour consistent with the existing Select Main Channel functionality. The user should be able to select from the applicable temperature channels.

- Ambient Temperature;
- Body Temperature;
- Monitor Temperature;
- Chamber Temperature;
- Hyperbaric Water Temperature.

The selected channel becomes the measurement used to determine whether the Temperature Swing has reached and stabilised at the requested condition.

Temperature Swing Sequence

The intended high-level sequence is:

- Start;
- apply delayed start if configured;
- perform normal program startup and begin the existing CSV recording;
- establish the requested pressure condition;
- configure/start the required temperature cabinet;
- send the requested temperature setpoint to the cabinet;
- monitor and control temperature and pressure during the swing;
- reach or pass the requested temperature;
- wait for temperature stabilisation;
- complete the Temperature Swing.

The detailed CODESYS state-machine design is for you to develop. Keep the implementation simple, readable and consistent with the existing programs.

Temperature Completion Requirement

The swing must not complete simply because the selected temperature briefly crosses the requested setpoint.

The selected monitoring channel must:

- reach or pass the requested setpoint in the correct direction;
- satisfy a temperature rate of change of less than 0.5 °C/min.

For heating, the acceptable extreme is:

- Setpoint to Setpoint + 11 °C.

For cooling, the acceptable extreme is:

- Setpoint to Setpoint - 11 °C.

Pressure Behaviour

The existing Test Pressure Start Dialog field should be used.

0 psi Temperature Swing

If Test Pressure = 0 psi, the program should operate as a zero-pressure temperature swing.

The default solenoid condition should be:

- Upstream closed;
- Downstream open.

This is intended to prevent pressure building within the test article as its temperature changes. Pressure-maintenance logic should be ignored for the duration of the swing.

Pressurised Temperature Swing

If Test Pressure > 0 psi, the program should first establish the requested test pressure. Investigate reuse of FB_Apply_Test_Pressure and the pressure-application approach used elsewhere in the project, including the existing 5 to 10% PR2 functionality where relevant.

Once pressure has been established, the program must continuously supervise pressure during the temperature swing and maintain it between 50% and 100% of the requested Test Pressure.

The exact cyclic control method is for you to investigate. A likely approach will involve the existing upstream and downstream solenoid controls. Reuse existing functions and patterns wherever practical.

End-of-Test Behaviour

Pressure

At successful completion of a pressurised Temperature Swing, do not automatically vent the pressure. Pressure release should remain a deliberate technician action so it can be carried out safely and slowly.

For a 0 psi Temperature Swing, leave the zero-pressure configuration in place.

Temperature Cabinet

For a normal hot or cold Temperature Swing, leave the temperature cabinet running at the requested setpoint after the program completes so that the technician can perform the next required test.

When the purpose of the swing is to return the valve to ambient conditions, the cabinet should be switched off when the user stops the test, once the agreed ambient condition has been met.

Do not use the whole API 6A ambient test range of 4 °C to 50 °C as an automatic cabinet-off rule.

A sensible ambient-detection method should be proposed before implementation, likely by comparing the relevant valve temperature with the measured Ambient Temperature using an appropriate tolerance.

Pressure Display / HMI Behaviour

Integrate the Temperature Swing into the existing Pressure Display rather than creating a separate temperature screen. Keep colours, cards, prompts and general behaviour consistent with existing functionality.

Prompt Bar

During the ramp, the prompt should clearly state the requested temperature and acceptable target range, for example:

- Temperature ramp to 120 °C. Target range: 120 °C to 131 °C.

Monitoring Channel

The selected Temperature Monitoring Channel should be highlighted using a coloured outline consistent with existing channel/status behaviour.

- use white while progressing towards the temperature target;
- use orange once the setpoint has been reached and the program is waiting for stabilisation.

Adjust the prompt text as necessary to make the current requirement clear.

Temperature Rate

Display the calculated temperature rate underneath the current value of the selected monitoring channel, similar to the stabilisation values used in existing programs.

- Example: 0.32 °C/min.

Chamber Temperature

Chamber Temperature should remain visible on the Pressure Display using the existing channel-card style.

Data Recording

Use the existing CSV/data logging system. Do not create a separate temperature logging system.

Recording should begin through the existing program Start behaviour and continue in the same way as other DLS programs.

Existing Functionality

Temperature Swing is an additional program. Existing programs must continue to operate normally.

Prefer reusing an existing function or creating a sensible reusable function block where appropriate. If a change to shared functionality appears necessary, confirm the intended change with TL before making a substantial architectural alteration.

Scope

In Scope

- new Temperature Swing program in the supplied CODESYS project;
- integration with the existing program selector;

- HTML HMI Start Dialog changes;
- positive and negative temperature setpoints;
- selectable temperature monitoring channel;
- existing delayed-start and CSV recording behaviour;
- 0 psi and pressurised temperature-swing variants;
- pressure establishment and 50-100% pressure supervision;
- API 6A temperature stabilisation detection;
- Pressure Display prompts, highlighting and temperature-rate indication;
- reuse of existing CODESYS and RnD functionality wherever practical;
- offline and live functional testing.

Out of Scope for Now

- modifying PR2 Dynamic Cycle Test into a temperature program;
- automatic execution of the complete API 6A three-thermal-cycle procedure;
- automatic chaining into pressure-hold programs;
- Test Profile/database schema changes;
- new CSV/data logging architecture;
- a new GitHub repository;
- replacement CODESYS projects;
- hardware modifications;
- extensive new fault-handling architecture before basic functionality is proven;
- universal hard-coded cabinet temperature limits;
- user-configurable API stabilisation or overshoot limits;
- artificial temperature ramp-rate control.

Development Approach

Stage 1 - Understand

Review the existing implementation before coding, including:

- program/state-machine architecture;
- pressure application/release functions;
- 5 to 10% pressure program behaviour;
- existing stabilisation calculations;
- Start Dialog and Pressure Display implementation;
- temperature cabinet communication from the previous project;
- the relevant API 6A requirements.

Stage 2 - Plan

Before substantial implementation, prepare a short proposed design covering:

- Temperature Swing state sequence;
- temperature-rate calculation;
- how pressure will be established;
- how 50-100% pressure will be maintained;
- existing functions to be reused;
- HMI changes required;
- OPC/new CODESYS variables required.

This does not need to be a large formal document. Review the proposed route with TL before moving into major implementation.

Stage 3 - Offline Development

Develop as much as practical without requiring a live cabinet. Build regularly, resolve errors as they arise, and commit/push sensible milestones to the agreed branch.

Stage 4 - Hardware Testing

When cabinet access becomes available, progressively test:

- cabinet communication;
- positive temperature swing;
- cooling temperature swing;
- negative setpoint where the selected cabinet supports it;
- temperature-channel selection;
- 0 psi behaviour;
- pressurised behaviour;
- temperature stabilisation detection;
- final program behaviour.

Do not try to prove every behaviour simultaneously on the first live test.

Stage 5 - Final Demonstration

Once individual functionality has been demonstrated, perform an end-to-end test from the HTML HMI using the agreed CODESYS project and GitHub branch.

When to Ask for Help

You are encouraged to investigate and solve technical problems yourself. However, please ask before proceeding if you are unsure whether something:

- changes the intended architecture;
- requires a different CODESYS project or GitHub repository/branch;
- affects existing programs or safety behaviour;
- changes physical hardware;
- contradicts the API specification;
- requires significant scope expansion;
- could make later integration substantially more difficult.

If an investigation produces an unexpected result, it is fine to stop and discuss it. Do not solve uncertainty by creating a completely separate implementation unless that has been agreed first.

Definition of Done

The Temperature Swing project is complete when a technician can:

- select Temperature Swing from the normal program selection interface;
- open a familiar Start Dialog and enter the normal test details;
- enter a positive or negative Temperature Setpoint;
- select the required Temperature Monitoring Channel;
- enter either 0 psi or a required Test Pressure;
- press Start once and have the existing CSV recording begin normally;

- have the DLS establish the correct cabinet and pressure conditions;
- have the cabinet automatically move towards the requested setpoint;
- have the DLS maintain the appropriate pressure condition during the swing;
- clearly see the requested target, relevant temperatures and °C/min rate on the HMI;
- have the DLS recognise when the selected temperature reaches/passes the requested setpoint and stabilises below 0.5 °C/min;
- complete without automatically venting a pressurised test;
- leave the cabinet at the requested condition ready for the next test, or correctly handle the agreed ambient-return behaviour;
- complete all of the above without adversely affecting existing DLS programs.

The final implementation should be demonstrated to TL from the agreed CODESYS project and GitHub branch.

Expected Approach

OJ should lead the investigation and technical implementation. The aim is not to be given every coding step, but to understand the existing system, reuse sensible patterns and propose a clean route forward.

Support will be available to steer the project, answer questions and review larger decisions. Ownership of the implementation should sit with OJ as much as possible.

The key principle is: investigate independently, but stay within the agreed architecture. If you are unsure about what should be achieved, whether a change affects the wider system, or whether you are moving outside the agreed scope, ask before going further.